Jenkins, pipeline and Kubernetes interview questions, answered with this project

Questions about Jenkins, CI/CD pipelines for tests, Docker and Kubernetes that come up in SDET and test automation interviews. Each answer has a short explanation and, where useful, an example, and ends with an **In the project** pointer to where this repository does it (or why it does not). Links go to github.com/niranjankagri/jenkins-kubernetes-pipeline.

Versions: Jenkins 2.580.1 LTS (JDK 21), Configuration as Code + Job DSL, kind 0.33, kubectl 1.37, Node 24, pytest 9.

Contents

1. Jenkins fundamentals (Q1-Q12)
2. Pipelines and the Jenkinsfile (Q13-Q28)
3. Jenkins as code and security (Q29-Q38)
4. Test reporting and quality gates (Q39-Q46)
5. Docker in pipelines (Q47-Q52)
6. Kubernetes for testers (Q53-Q66)
7. Scenarios and troubleshooting (Q67-Q76)
8. Quick reference

---

1. Jenkins fundamentals

Q1. What is Jenkins?

An open-source automation server, used mostly for CI/CD: it runs jobs (builds, tests, deployments) when code changes, on a schedule or by hand, and keeps their logs, results and artifacts. Its power comes from about 2,000 plugins (git, pipelines, test reports, Kubernetes, credentials...).

In the project: a Jenkins 2.580.1 LTS controller in Docker with ten plugins.

Q2. Controller and agents: what is the difference?

The **controller** serves the UI and API, stores configuration and history, and schedules builds. **Agents** run the builds (on other machines, in Docker containers or in Kubernetes pods). Builds should not run on the controller in production: it is a security risk (build code can read Jenkins' files) and a capacity risk.

In the project: builds run on the controller with 2 executors, as a documented simplification (`casc.yaml`, README "Known limitations").

Q3. What is an executor?

A slot for one build at a time on a node. A node with 2 executors runs two builds in parallel.

In the project: `numExecutors: 2` in `casc.yaml`; parallel stages inside one build do not need extra executors.

Q4. Freestyle job vs Pipeline job?

Freestyle jobs are configured in the UI with build steps; hard to review, version or reuse. **Pipeline** jobs are defined in code (a Jenkinsfile in the repository): versioned with the application, reviewed in pull requests, with stages, parallelism and conditions. Multibranch pipelines create one job per branch automatically.

In the project: a pipeline job whose Jenkinsfile lives in the repository.

Q5. Declarative vs scripted pipeline?

Declarative (`pipeline { ... }`) has a fixed structure (agent, stages, steps, post, options), is validated before running and is easier to read. **Scripted** (`node { ... }`) is plain Groovy: more flexible, easier to get wrong. Declarative with small `script {}` blocks is the usual choice.

In the project: declarative; `script {}` only in the quality gate and the teardown condition.

Q6. What triggers a build?

SCM changes (webhooks from GitHub/GitLab, or polling), schedules (`cron` in `triggers`), other jobs (upstream), manual builds and the REST API.

In the project: started by hand or through the REST API (`scripts/run-jenkins-build.sh`); CI uses the API.

Q7. Webhook or polling?

A webhook: the git server tells Jenkins about a push immediately. Polling asks the git server on a schedule: delayed and wasteful. Polling remains a fallback when Jenkins is not reachable from the git server.

In the project: neither (local demo, repository mounted from disk).

Q8. What is a workspace?

The folder where a build checks out the code and works. It persists between builds of the same job on the same node unless cleaned.

In the project: the Python virtual environment in the workspace is reused between builds (faster); `post { cleanup }` removes the report folders so old reports never count.

Q9. What are artifacts, and why archive them?

Files a build produces (packages, reports, logs). Archiving keeps them with the build after the workspace changes.

In the project: `archiveArtifacts` keeps the JUnit XML, the HTML reports and the port-forward logs.

Q10. How do you keep Jenkins from filling its disk?

Build discarders (`buildDiscarder(logRotator(numToKeepStr: '20'))`), workspace cleanup, limits on archived artifacts, and removing old images and containers on agents.

In the project: log rotation in both the job (Job DSL) and the Jenkinsfile options.

Q11. What is Blue Ocean, and what replaces it?

Blue Ocean was a UI for pipelines; it is no longer developed. The **Pipeline Graph View** plugin is the modern stage graph in the classic UI.

In the project: Pipeline Graph View (`docs/images/jenkins-stages.png`).

Q12. Jenkins vs GitHub Actions vs GitLab CI?

Jenkins: self-hosted, extremely flexible, plugin ecosystem, any environment, but you run and upgrade it. GitHub Actions / GitLab CI: hosted with the code, YAML, less to operate, marketplace actions. Many companies run both. The skills transfer: stages, artifacts, secrets, caching, parallelism.

In the project: both: Jenkins runs the pipeline; GitHub Actions tests the scripts and runs Jenkins itself.

---

2. Pipelines and the Jenkinsfile

Q13. What does a declarative pipeline look like?

```
pipeline {
  agent any
  options { timeout(time: 30, unit: 'MINUTES') }
  stages {
    stage('Test') { steps { sh 'npm test' } }
  }
  post { always { junit 'reports/*.xml' } }
}
```

In the project: `Jenkinsfile`, eight stages.

Q14. What is `agent`, and which kinds exist?

Where the pipeline (or a stage) runs: `any`, a `label`, a `docker` image, a `dockerfile`, or a `kubernetes` pod template (Kubernetes plugin). A stage can override the pipeline's agent.

In the project: `agent any` (the controller).

Q15. How do you run stages in parallel?

A stage with a `parallel { stage(...) stage(...) }` block. Use it for independent work; add `failFast true` to stop the others when one fails.

In the project: "Checks" (unit tests || lint) and "API tests" (smoke || regression).

Q16. How do you skip a stage conditionally?

`when { ... }`: `branch 'main'`, `expression { params.X == 'y' }`, `environment name: ..., value: ...`, `changeset '**/*.ts'`, combined with `allOf` / `anyOf` / `not`.

In the project: Regression runs only `when { expression { params.TEST_SUITE == 'all' } }`.

Q17. How do parameters work?

`parameters { choice(...) booleanParam(...) string(...) }` adds inputs to "Build with Parameters"; the values are in `params.NAME`. The job learns the parameters on its first run; after that, the REST API needs `buildWithParameters`.

In the project: `TEST_SUITE` and `KEEP_NAMESPACE`; the API script handles both cases.

Q18. What are `options` used for?

Build-wide settings: `timeout`, `timestamps`, `retry`, `buildDiscarder`, `disableConcurrentBuilds`, `skipDefaultCheckout`, `quietPeriod`.

In the project: timestamps, a 30-minute timeout, log rotation, no concurrent builds (one shared cluster).

Q19. Explain the `post` conditions.

After the stages (or a stage): `always`, `success`, `failure`, `unstable`, `changed`, `aborted`, `fixed`, `regression`, and `cleanup` (last, regardless of result). Reports go in `always`; notifications in `success` / `failure`; deleting temporary things in `cleanup`.

In the project: reports and teardown in `always`, notification in `success`, a message in `failure`, report folders removed in `cleanup`.

Q20. How do environment variables work?

`environment { NAME = "value" }` at pipeline or stage level; built-ins like `BUILD_NUMBER`, `WORKSPACE`, `GIT_COMMIT`, `JOB_NAME`. In `sh` steps they are shell variables (`$NAME`); in Groovy `env.NAME`.

In the project: `IMAGE`, `NAMESPACE`, `KUBECONFIG`, `PYTHON` built from `BUILD_NUMBER` and `WORKSPACE`.

Q21. Single quotes vs double quotes in `sh` steps?

Double-quoted Groovy strings are interpolated by Groovy before the shell sees them; single-quoted ones are passed to the shell, which expands `$VAR`. Use single quotes for secrets (Groovy interpolation of a secret leaks it into the step's recorded command and triggers a warning) and for anything the shell should expand.

In the project: single quotes everywhere except where a Groovy value (`GIT_COMMIT.take(7)`) is needed.

Q22. How do you stop a stage from running forever?

`timeout` in `options` (whole build) or around steps (`timeout(time: 5, unit: 'MINUTES') { ... }`); and timeouts inside the scripts (`kubectl ... --timeout=120s`).

In the project: a 30-minute build timeout; rollout 120 s, smoke Job 90 s, HTTP 10 s in the tests.

Q23. How do you retry a flaky step, and should you?

`retry(3) { ... }` or the `retry` option. Retry infrastructure steps that can fail transiently (downloads, a registry); do not hide flaky tests behind retries.

In the project: no retries; waits are explicit (rollout status, health loop before tests).

Q24. What are stash and unstash?

Carry files between stages that run on different agents. Not needed when all stages run in one workspace.

In the project: not needed (one workspace).

Q25. What is a shared library?

A git repository with reusable pipeline code (`vars/` steps, `src/` classes), loaded with `@Library('name')`. It keeps many pipelines short and consistent.

In the project: not used: the reusable logic lives in shell scripts, which also run outside Jenkins.

Q26. Why put logic in scripts instead of the Jenkinsfile?

Scripts run and debug the same way locally, in Jenkins and in any other CI; Groovy in the Jenkinsfile runs only in Jenkins (and in its sandbox). The Jenkinsfile then reads like the process.

In the project: six scripts, called by both the Jenkinsfile and GitHub Actions.

Q27. What is "Replay" and "Restart from Stage"?

Replay re-runs a build with an edited Jenkinsfile (for trying fixes without committing). Restart from Stage re-runs a declarative pipeline from a chosen stage with the same inputs.

In the project: both available in the build menu (see the screenshots); fixes were committed instead, so history shows them.

Q28. What is the Groovy sandbox?

Pipeline code runs in a sandbox that only allows approved methods. Anything else needs an administrator's approval ("In-process Script Approval"). Keep pipelines simple instead of approving more.

In the project: CI's Jenkins rejected `Collection.sum()`; a plain loop replaced it.

---

3. Jenkins as code and security

Q29. What is Jenkins Configuration as Code (JCasC)?

A plugin that configures Jenkins from YAML at start-up: security realm, users, permissions, credentials, tools, global settings, clouds. The configuration lives in git and is applied the same way every time.

In the project: `jenkins/casc.yaml`.

Q30. How are jobs created as code?

The **Job DSL** plugin (often from JCasC's `jobs:` section) defines jobs in Groovy; or multibranch/organisation folders discover repositories with Jenkinsfiles automatically.

In the project: a `pipelineJob` in Job DSL inside `casc.yaml`.

Q31. How do you install plugins reproducibly?

A `plugins.txt` and `jenkins-plugin-cli` in the image build. Pin versions for reproducible images and upgrade them deliberately.

In the project: `plugins.txt` (unpinned, noted as a limitation).

Q32. How do you handle secrets in Jenkins?

Store them in the credentials store (from JCasC with values from the environment or a secret manager), bind them to steps with `withCredentials`, use single-quoted `sh` strings, and let Jenkins mask them. Never in the Jenkinsfile or the repository.

In the project: `notify-token` in `casc.yaml` (value from `NOTIFY_TOKEN`), bound in the `success` block; the log shows only its length.

Q33. How is Jenkins secured?

A security realm (local users, LDAP, SSO), an authorisation strategy (matrix or role-based), no anonymous access, CSRF protection, agents instead of controller executors, minimal plugins kept up to date, HTTPS.

In the project: local admin from the environment, no signup, no anonymous read, CSRF on (the API script fetches a crumb).

Q34. What is the CSRF crumb?

A token Jenkins requires on POST requests together with the session that issued it, against cross-site request forgery. API clients fetch it from `/crumbIssuer/api/json` (or use an API token, which is exempt).

In the project: `run-jenkins-build.sh` gets the crumb with a cookie jar and posts with both.

Q35. Why is the Docker socket in a Jenkins container a risk?

Access to the Docker socket is access to the host: a build could start a privileged container and read anything. Acceptable on a local demo machine; on shared Jenkins use rootless builds (Kaniko, BuildKit) in agents.

In the project: used and documented as a limitation.

Q36. What does relaxing the CSP for HTML reports mean?

Jenkins serves archived files with a strict Content Security Policy so reports cannot run scripts. Reports that need JavaScript (pytest-html builds its table with it) stay empty. Relaxing the CSP is a trade-off; alternatives are static reports or a separate report server.

In the project: relaxed for the local demo, with a comment in `docker-compose.yml`.

Q37. How would you upgrade Jenkins safely?

Build a new image with the new LTS and plugins, start it against a copy of the data (or a test instance), run the pipelines, then switch. With everything as code, a failed upgrade is a rollback to the previous image.

In the project: the image and configuration are in git; CI builds and runs a fresh Jenkins on every push.

Q38. How do you back up Jenkins?

`JENKINS_HOME` holds everything (config, jobs, builds). With configuration as code, the important part is in git; back up build history and credentials separately if they matter.

In the project: a named Docker volume (`jenkins_home`); configuration in git.

---

4. Test reporting and quality gates

Q39. How do test results appear in Jenkins?

The `junit` step reads JUnit XML: Jenkins shows a test tree, failures with messages, history per test, a trend chart, and marks the build UNSTABLE when tests fail.

In the project: unit and API reports recorded; trend chart on the job page.

Q40. Why JUnit XML?

Every test tool can write it (pytest `--junitxml` , `node --test --test-reporter=junit` , Maven Surefire, Cypress reporters), and every CI tool can read it.

In the project: Node's and pytest's JUnit output.

Q41. How do you publish HTML reports?

The HTML Publisher plugin (`publishHTML`) adds a link per report to the build, keeping old builds' reports with `keepAll` .

In the project: "API tests: smoke" and "API tests: regression" links.

Q42. What is a quality gate, and what did you put in yours?

A rule that decides whether a build may continue or pass. Mine: no failed tests, and at least a minimum number of tests ran. The second rule catches suites that silently ran nothing (a wrong marker, a collection error).

In the project: the "Quality gate" stage.

Q43. Why did Jenkins once show 42 tests instead of 21?

The gate called `junit` again on files that the stages had already recorded; Jenkins added them a second time. Each stage now records its report once and keeps the returned summary, and the gate adds the summaries up.

In the project: the trend chart shows the drop from 42 to 21 at build #6.

Q44. Failed tests: FAILURE or UNSTABLE?

By convention, compile and infrastructure errors are FAILURE and failed tests UNSTABLE. Many teams fail the build on any failed test so nothing gets merged or deployed. Decide explicitly.

In the project: a failing pytest run fails its stage (non-zero exit), so the build is FAILURE.

Q45. How do you make results visible to the team?

Notifications (chat, e-mail) on failure and on fixed builds, badges, dashboards, and test reports linked from pull requests.

In the project: a notification step with a masked token as the hook point; badges on the README.

Q46. How do you track flaky tests?

Test history across builds (Jenkins shows age and changes per test), plugins or tools that flag flip-flopping tests, and a quarantine list with owners. Fix or delete; never ignore.

In the project: tests are deterministic (own data per test, explicit waits); history is visible in Jenkins.

---

5. Docker in pipelines

Q47. Docker-in-Docker vs Docker outside of Docker?

DinD runs a separate Docker daemon inside a privileged container. **DooD** mounts the host's Docker socket so the container's `docker` CLI uses the host's daemon. DooD is simpler and shares the image cache; both give broad host access.

In the project: DooD (`/var/run/docker.sock` mounted).

Q48. Why did the socket need a group setting?

The socket belongs to a group on the host (its ID differs per machine). The `jenkins` user must be in that group to use it; Compose's `group_add` with the host's group ID does it.

In the project: `group_add: ['${DOCKER_GID:-0}']`; CI passes `stat -c %g /var/run/docker.sock`.

Q49. How do you keep an image small and safe?

A slim base (alpine/slim), only what runs (no build tools), a non-root user, pinned base versions, a health check, and no secrets in layers.

In the project: `service/Dockerfile`: node:24-alpine, no npm install, `USER node`, health check.

Q50. How do you tag images in a pipeline?

Unique and traceable: the build number and/or the commit; never only `latest`.

In the project: `room-booking:build-<n>`; the Deployment gets the build number and the commit.

Q51. How do you install tools in a Jenkins image?

Copy static binaries from official images (multi-stage `COPY --from`), use the OS packages for runtimes, and verify downloads with published checksums.

In the project: docker CLI and Node copied from their images; kind and kubectl checked against SHA-256 sums.

Q52. What does `docker compose up --wait` do?

Starts the services and waits until their health checks pass (or fails). Useful in scripts and CI.

In the project: Jenkins has a health check on `/login`; CI waits for it.

---

6. Kubernetes for testers

Q53. What are the main Kubernetes objects?

- **Pod**: one or more containers scheduled together.
- **Deployment**: keeps N identical pods running and rolls out new versions.
- **Service**: a stable name and address for a set of pods.
- **Namespace**: a scope for names and policies.
- **Job**: runs pods to completion (a task, a test).
- **ConfigMap / Secret**: configuration and credentials.

In the project: Deployment, Service, Job, Namespace.

Q54. What is kind, and why use it?

"Kubernetes in Docker": cluster nodes as Docker containers. A real control plane for CI and laptops, created in about a minute, deleted in seconds. Alternatives: minikube, k3d, Docker Desktop's Kubernetes.

In the project: a kind cluster named `qa`, created once and reused.

Q55. How does an image get into a kind cluster?

`kind load docker-image <image>` copies it into the nodes; the Deployment uses `imagePullPolicy: Never` (or `IfNotPresent`). Real clusters pull from a registry.

In the project: loaded after each build.

Q56. Readiness vs liveness probes?

Readiness: is the pod ready for traffic? Until it is, the Service sends nothing to it and a rollout waits.
Liveness: is it still healthy? If not, Kubernetes restarts the container. (Startup probes protect slow starters.)

In the project: both on `/health`.

Q57. How do you know a deployment finished?

`kubectl rollout status deployment/<name> --timeout=...` waits until the new pods are ready and the old ones are gone, and fails on timeout.

In the project: `deploy.sh`; on failure it prints pods and events.

Q58. How do you debug a failed rollout?

`kubectl get pods`, `kubectl describe pod` (events: image pull errors, probe failures, scheduling), `kubectl logs` (also `--previous` after a crash), `kubectl get events --sort-by=.lastTimestamp`.

In the project: the script prints pods and recent events.

Q59. Why a namespace per build?

Isolation: builds never see each other's objects, names do not clash, and cleanup is one command. It is the cheapest form of "ephemeral environment".

In the project: `qa-<build number>`, deleted in `post`.

Q60. How do tests reach a service in the cluster?

From inside: the Service's DNS name (`http://room-booking:8080`, or `room-booking.qa-7.svc.cluster.local`). From outside: `kubectl port-forward`, a NodePort, a LoadBalancer or an Ingress.

In the project: the smoke Job uses the DNS name; the API tests use port-forward.

Q61. Why run a smoke test as a Kubernetes Job?

It tests the deployment from where other services call it: inside the cluster, through DNS and the Service. `kubectl wait --for=condition=complete` turns it into a pass/fail step.

In the project: `k8s/smoke-job.yaml`, `backoffLimit: 0` so a failure is not retried quietly.

Q62. How do you prove the right version is deployed?

Expose it (`/version`, a header, a label), set it from the pipeline, and assert it in a smoke test. Catches stale pods, wrong tags and failed rollouts that kept the old version.

In the project: `APP_VERSION` = build number; checked by the smoke Job and `test_the_expected_build_is_deployed`.

Q63. What are resource requests and limits?

Requests: what the scheduler reserves. Limits: the maximum (memory over the limit kills the container, CPU is throttled). They make tests reproducible and protect neighbours.

In the project: 50m CPU / 64 Mi requested, 128 Mi memory limit.

Q64. How does a pod stop?

Kubernetes sends SIGTERM, waits for the grace period (30 s by default), then SIGKILL. Apps should stop accepting work and finish requests on SIGTERM; otherwise rollouts drop requests.

In the project: the service closes its server on SIGTERM.

Q65. How does Jenkins reach a kind cluster from inside a container?

The kubeconfig points to `127.0.0.1:<port>` on the host, which is not the container. The Jenkins container joins Docker's `kind` network and uses `kind get kubeconfig --internal` (the control-plane container's name).

In the project: `kind-up.sh` detects `/.dockerenv` and does exactly that.

Q66. How would Jenkins use Kubernetes in production?

The Kubernetes plugin starts a pod per build as the agent (a `podTemplate` with containers for each tool), so builds scale with the cluster and run isolated; plus deployments to real clusters with RBAC and a registry.

In the project: not yet (builds on the controller); listed as the next step.

---

7. Scenarios and troubleshooting

Q67. "Couldn't find any revision to build." What do you check?

The repository URL, the branch spec (`*/main`), credentials, and whether the fetch really brought refs. In this project the cause was a **shallow** CI checkout: git refuses to update refs from it with only a warning.

In the project: `fetch-depth: 0` for the Jenkins job in CI.

Q68. The REST API answers 400 when starting a job. Why?

A job with parameters must be started with `/buildWithParameters` ; a job without, with `/build` . Missing crumb or permissions give 403.

In the project: the script tries `buildWithParameters` , then `build` .

Q69. kubectl in the pipeline returned a Jenkins login page. What happened?

Without a kubeconfig, kubectl defaults to `localhost:8080` , which inside the Jenkins container is Jenkins. The teardown ran in a build that stopped before the cluster stage.

In the project: `teardown.sh` skips when there is no kubeconfig.

Q70. A test report step fails with "No such file". Causes?

The tests did not run (an earlier failure), wrote elsewhere, or the reporter could not create the folder. Node's JUnit reporter does not create folders.

In the project: `mkdir -p reports` before the unit tests.

Q71. A test fails in the pipeline but passes locally. Steps?

Read the report and log; compare versions (tool, runtime, image), configuration (environment variables, URLs), data (shared state, order) and timing (slower machine, missing waits); reproduce with the pipeline's script locally.

In the project: the scripts can be run by hand with the same arguments.

Q72. How do you make the pipeline fail fast?

Cheap checks first (lint, unit tests) before slow ones (image build, deploy, end-to-end tests), parallel independent checks, and `failFast` where useful.

In the project: a broken booking rule failed at unit tests in the first stage; nothing was built or deployed.

Q73. How do you speed up a slow pipeline?

Measure stage times; cache dependencies and images; reuse expensive environments (a cluster) but isolate per build (a namespace); parallelise; run the full regression on main and a smoke subset on pull requests.

In the project: cluster reuse (2 min → about 30 s), cached venv, parallel stages, `TEST_SUITE=smoke`.

Q74. How do you test a Jenkinsfile?

Lint it (the declarative linter via the API or CLI), unit-test shared libraries (JenkinsPipelineUnit), and run it for real: here, CI starts a real Jenkins and requires a passing build.

In the project: the "Jenkins runs the Jenkinsfile" CI job.

Q75. Describe this project in two minutes.

"Jenkins in Docker, configured entirely as code: plugins in the image, JCasC for users, security and a credential, and the job created by Job DSL. The Jenkinsfile runs unit tests and lint in parallel, builds a Docker image, deploys it to a kind Kubernetes cluster in a namespace per build, runs a smoke Job inside the cluster, then API smoke and regression tests in parallel, and a quality gate that also catches suites that ran too few tests. The logic is in shell scripts that GitHub Actions also runs, and CI starts a real Jenkins on every push and fails unless the pipeline passes. Builds take about 30 seconds once the cluster exists."

In the project: README.

Q76. What would you add next?

Kubernetes pod agents instead of controller executors, a registry, pinned plugins, a shared library, webhooks and multibranch pipelines for pull requests, and notifications to chat.

In the project: README "Known limitations", design doc section 8.

---

8. Quick reference

Need	Code or command
parallel stages	`stage('X') { parallel { stage('a') {...} stage('b') {...} } }`
condition	`when { expression { params.TEST_SUITE == 'all' } }`
record tests	`def summary = junit 'reports/*.xml'` (`summary.totalCount`, `failCount`, `skipCount`)
HTML report	`publishHTML(target: [reportName: ..., reportDir: ..., reportFiles: ..., keepAll: true])`
secret	`withCredentials([string(credentialsId: 'id', variable: 'TOKEN')]) { sh 'use "$TOKEN"' }`
always / last	`post { always {...} cleanup {...} }`
start job via API	crumb from `/crumbIssuer/api/json`, POST `/job/<name>/buildWithParameters`
kind cluster	`kind create cluster --name qa`; `kind load docker-image img --name qa`
deploy and wait	`kubectl apply -n ns -f ...`; `kubectl rollout status deployment/x -n ns --timeout=120s`
run a Job	`kubectl wait job/smoke -n ns --for=condition=complete --timeout=90s`
reach a Service	`kubectl port-forward -n ns service/x 18080:8080`
debug	`kubectl describe pod`, `kubectl logs`, `kubectl get events --sort-by=.lastTimestamp`